

# Traffic Sign Recognition

Team 9: Ahn Kun Woo, Ryan Ng Jek Hian, Shyann Loh, Tai Zong Jie

## Abbreviations

DL – Deep Learning, ML – Machine Learning, TSR – Traffic Sign Recognition, CNN – Convolutional Neural Network, KNN – K-Nearest Neighbours, SVM – Support Vector Machines

## Introduction

The ability to recognise and classify traffic signs is crucial in improving autonomous driving and road safety. This project leverages DL and ML algorithms - trained on a dataset of approximately 5000 traffic sign images - to classify traffic sign images into 23 distinct classes. These images capture diverse real-world conditions, including variations in lighting, obstructions, and sign deterioration, all of which contribute to challenges in achieving accurate classification. This report explores a combination of CNN-based models and traditional ML models, utilising feature extraction and pre-trained knowledge for classification. To improve the image classification accuracy, various hyperparameters were experimented and CNN architectures were fine-tuned, effectively enhancing the model's performance in classifying traffic signs under diverse conditions.

## Related Works

One of the key challenges in TSR is the need for a large training set to ensure effective classification across varying conditions. Khan et al. (2023) highlights the potential of using transfer learning with pre-trained models such as ResNet, which has been trained on extensive datasets, to enhance feature extraction and improve classification accuracy. However, a drawback is the high computational demand commonly associated with large pre-trained models, reducing the model's feasibility for real-time classification in autonomous vehicles. To address this issue, Khan mentioned the use of lightweight CNN model specifically trained for the classification task rather than relying on pre-trained models, an approach which we adopted in our methodology. Fleyeh (2008) highlights the significant influence of weather variability on classification accuracy, as colour information can shift considerably under different weather conditions, thereby impacting model performance. To mitigate this effect, the report highlighted the need for training dataset to include images of traffic signs under different weather conditions to ensure a representative range of visual variations.

## Dataset

The dataset used in this project comprises 4,896 images of traffic signs, each belonging to one of 23 distinct classes, as illustrated in Figure 1.



Figure 1: Photographs of Various Traffic Signs

The dataset covers a range of visual characteristics such as variations in lighting, backgrounds, and angles. This provides robustness for training and testing ML models to recognize and classify signs accurately across different scenarios. Each class is uniquely represented by its shape, colour, and symbol, which are critical features for model differentiation.

However, there are limitations that affect the dataset's effectiveness. These include class imbalance, where some traffic sign classes are under-represented, potentially biasing the model towards more frequently occurring classes. Additionally, the images vary in size, requiring standardisation of dimensions for consistent model input. Another limitation is the lack of explicit weather condition filters that impact sign visibility. As such, we augmented our dataset by applying simulated weather filters, enabling it to maintain accuracy across a variety of environmental scenarios.

## Preprocessing

Various image dimensions were tested (32x32, 64x64, and 224x224), with 64x64 yielding the highest accuracy. The images were normalised to prevent issues arising from varying image dimensions ensuring the pixel values were standardised to the range of 0 and 1. Allowing for faster model convergence and reducing sensitivity to lighting and colour variations, making the model more robust to diverse image conditions.

To enhance our model's generalisation to unseen data and reduce overfitting, we introduced an external dataset as an additional test set. This dataset includes balanced image classes with five supplementary images per class, enabling a thorough evaluation of the model's ability to perform on new, diverse data.

For feature selection, we created our own pooling layer by creating a 4x4 matrix to drag across each image, extracting the maximum value from each patch of the feature map. We applied this on our traditional ML models to extract the

significant features from the input images and reduce the number of features to help control overfitting, such that the models perform better on unseen data.

## Methodology

To improve the model's capacity to generalise to unseen images, a series of data augmentation techniques were applied, including rotations, shifts, zooms, and brightness adjustments. Additionally, we introduced custom weather filters as seen in Figure 2, to simulate environmental conditions such as fog and rain, preparing the model for complex realistic scenarios not included in the training set. This augmentation approach addresses the limitations in prior studies, where models often faltered in unpredictable conditions, hence enhancing the model's robustness.



Figure 2: Dataset put under various weather filters

We modelled the data using various DL frameworks and traditional ML models before comparing their performance. Linear and Logistic Regression were not appropriate for the task as the former assume linearity while the latter is used for binary classification.

Firstly, we used the CNN model. CNN is a DL model specifically optimised for image analysis, utilising layers of convolution, pooling, and nonlinear activations to capture complex spatial structures within visual data. Through sequential convolutional layers, CNNs autonomously learn and extract hierarchical features, from simple edges and textures to more intricate patterns, allowing the model to differentiate subtle visual characteristics across classes. This feature makes CNNs particularly effective for image classification tasks.

We also explored ResNet50, which is a deep CNN model that has been pre-trained on large datasets and is known to be effective for image classification. It utilises a "residual learning" approach, to address the challenge of vanishing gradients in very deep networks. Leveraging a pre-trained model enables transfer learning, allowing the model's pre-learned weights to serve as a foundation for feature extraction, reducing the need for extensive training data.

Next, we used RF classifier, which utilises ensemble learning by combining the outputs of multiple decision trees trained on the same dataset. This approach averages the predictions from each tree to produce a final output. This significantly reduces the variance inherent in individual decision trees and reduces the bias of each tree.

Thirdly, we used KNN, which is a non-parametric and instance-based algorithm. Since KNN does not make assumptions about the underlying data distribution, it can adapt well to complex patterns such as the variety of shapes and colour patterns observed in the traffic sign dataset.

Finally, we employed SVM due to their effectiveness in high-dimensional spaces and robustness to overfitting. We extracted features using techniques such as Histogram of Oriented Gradients (HOG). These features were fed into the SVM classifier, which was trained to distinguish between various traffic sign categories. A radial basis function (RBF) kernel was used to handle the non-linear relationships in the data, optimising the hyperparameters through grid search and cross-validation to achieve the best performance.

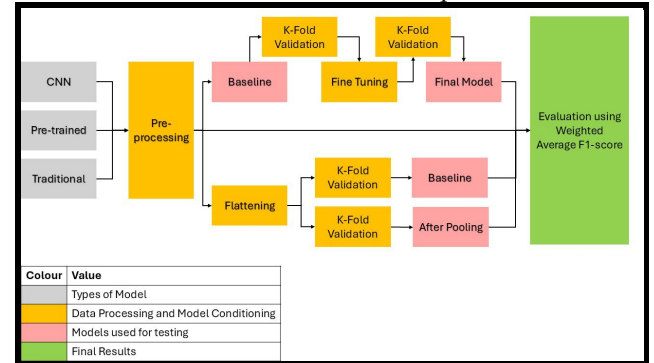


Figure 3: Flowchart for Methods

Figure 3 illustrates the flow of our approach to the problem. The dataset was put through a series of pre-processing steps to prepare it for training using the aforementioned models. Next, we used K-Fold Validation for conditioning the models and tested against our test datasets, after which we derived the results and evaluated it using Weighted Average F-1 Score.

## Results

To evaluate the performance of our models, we focused on weighted average F1-score as our primary metric. This metric ensures a balance between precision and recall, which is important for imbalanced datasets. The weighted average F1-score also assigns a greater weight to classes with higher sample count, providing a more accurate approximation of the overall model performance.

To optimise the CNN model, a Grid Search was implemented to evaluate a range of hyperparameter combinations, including batch size, epochs, optimiser, and dropout rates. The top 3 hyperparameter configurations were identified and validated against the validation set. The parameters yielding the highest validation accuracy is shown in Figure 4. This result indicates the optimal model parameters for maximising performance across all classes.

| Rank | Dropout Rate | Batch Size | Epochs | Optimizer | Validation Accuracy |
|------|--------------|------------|--------|-----------|---------------------|
| 1    | 0.3          | 32         | 10     | adam      | 1                   |
| 2    | 0.3          | 32         | 10     | rmsprop   | 0.9974              |
| 3    | 0.5          | 32         | 10     | rmsprop   | 0.9974              |

Figure 4: Table of Top 3 hyperparameters

To improve the CNN's model generalisation and prevent overfitting, data augmentation techniques were applied.

Transformations include rotation, width and height shifts, shearing, zooming in, filling in empty spaces using the nearest pixel values, and using a randomised set of weather filters. This enabled the model to learn from a more varied dataset and better adapt to variations in the input data. To assess the effectiveness of fine-tuning compared to the baseline CNN model, Local Interpretable Model-Agnostic Explanations (LIME) was used to visualise and interpret each prediction. LIME perturbs the input data and analyses the model's responses, identifying which features significantly impact each decision. By applying LIME before and after fine-tuning, we observed that the model was more effective in distinguishing subtle differences among visually similar classes. This comparison revealed that the fine-tuned model targeted important features more accurately, proving that the tuning process was essential in improving classification accuracy. Some examples of LIME are shown in Figure 5, whereby the model's responses are demarcated by the yellow line.

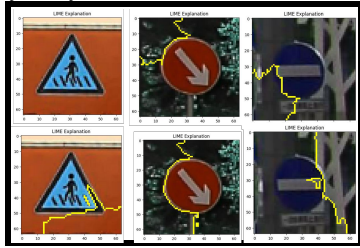


Figure 5: Visualisation using LIME

To evaluate the performance of each model and its ability to generalise, 5-fold cross-validation was conducted to generate the average validation accuracy and loss. This approach reduces overfitting and provides an estimate of the model's capacity to generalise to unseen data, as results were averaged across the folds, accounting for variability within the dataset.

Inspired by CNN's pooling layer, we added custom pooling layers to traditional ML models to improve feature selection. We evaluated the impact of pooling layers on our models by comparing the weighted F1 average before and after their application. However, this led to a drop in weighted F1 scores in both internal and external tests, seen in Figure 6. The decline is due to traditional ML models operating on a single layer of input features, lacking the multiple layers needed to build hierarchies, resulting in information loss and reduced performance.

| Model | Internal       |               | External       |               |
|-------|----------------|---------------|----------------|---------------|
|       | Before Pooling | After Pooling | Before Pooling | After Pooling |
| RF    | 0.98           | 0.97          | 0.70           | 0.66          |
| k-NN  | 0.78           | 0.74          | 0.57           | 0.45          |
| SVM   | 0.95           | 0.86          | 0.66           | 0.65          |

Figure 6: Weighted Average F1 scores of traditional ML models

After training each model, we evaluated the performance of the models using the weighted average F1-score, against both internal and external test sets as shown in Figures 7 and 8 respectively. Results showed that the CNN model outperformed the others for both test sets. This can be attributed to CNN's specialised architecture, which is well-suited for image data. Unlike traditional models, which rely on manually engineered features, CNN automatically learns spatial patterns through its convolutional layers. Moreover, traditional models treat each pixel independently and lack the structural adaptability to understand spatial relationships within images. While the pre-trained model, ResNet50, is trained on a large dataset and is known to be effective for image classification tasks, it performed the worst amongst all the models. ResNet50 excels in general object recognition due to its pre-training on large datasets but lacks the specificity needed to accurately classify traffic signs. Amongst the traditional ML models, RF and SVM outperformed KNN quite significantly. This can be attributed to k-NN inability to handle large datasets effectively as it requires calculating the distance between the test sample and every training sample to make a prediction.

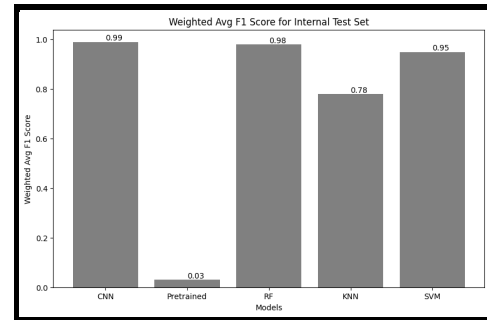


Figure 7: Model performance for Internal Test Set

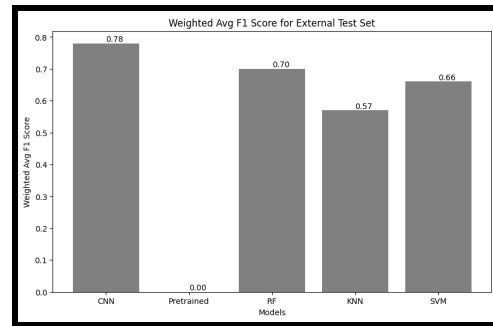


Figure 8: Model performance for External Test Set

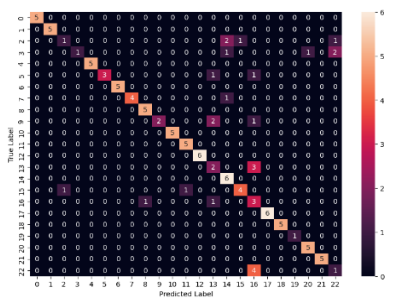
## Final Model

As such, the fine-tuned CNN model was selected as our final model. Additional in-depth analysis of the model was conducted, using 5-fold cross validation. The average validation accuracy obtained was 0.9893, while average validation loss was 0.0337. The performance of the fine-tuned model for internal and external datasets are shown in Figures 9 and 10 respectively. A confusion matrix was plotted to visualise the prediction for each class for the

external dataset, to identify the classes that the model struggled with. To ensure the model is not overfitted, the graphs of validation loss and accuracy of the training and validation datasets were plotted as shown in Figures 11 and 12 respectively. Since both graphs converge, it is evident that the model is not overfitted.

| Accuracy: 0.99   | Precision | Recall | F1-Score |
|------------------|-----------|--------|----------|
| Weighted Average | 0.99      | 0.99   | 0.99     |

Figure 9: Results of fine-tuned CNN against internal dataset



| Accuracy: 0.78   | Precision | Recall | F1-Score |
|------------------|-----------|--------|----------|
| Weighted Average | 0.83      | 0.79   | 0.78     |

Figure 10: Results of fine-tuned CNN against external dataset

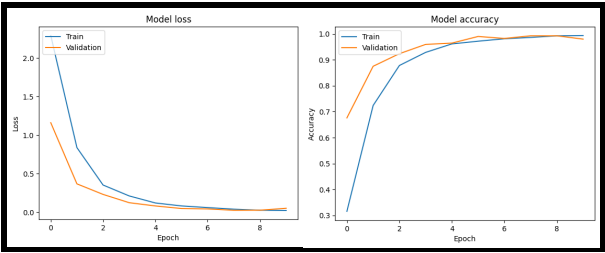


Figure 11: Fine-tuned CNN loss graph      Figure 12: Fine-tuned CNN accuracy graph

Limitations



Figure 13: Misclassified images for final CNN model (External Set)



Figure 14: LIME on misclassified external image (Left: Extreme darkness, Right: Dominant Features)

Our investigations concluded that our CNN model had difficulties handling images at very dark or bright conditions, as seen in the 4<sup>th</sup> and last images of Figure 13. This indicates that the data augmentation process used did not adequately represent these end-range conditions. Adding more augmentation techniques to cover extreme lighting conditions could improve the model's robustness. In addition, the model focuses on the dominant features in images, as seen in Figure 14. This sensitivity to visually prominent features means that larger traffic signs are more accurately identified, while less visually dominant signs tend to be overlooked. This could lead to misclassification of smaller signs, such as those with lower visual contrast or smaller physical size.

Discussions

Overall, our model shows promising recognition capabilities, particularly in various weather conditions due to data augmentation with weather filters, often outperforming human recognition in these scenarios. However, humans can quickly learn new traffic signs using prior knowledge, while our model needs retraining to adapt to novel signs, limiting its flexibility. One key challenge in our TSR model is its limited interpretability. Trained on only 23 types of traffic signs, our model does not account for variations in design or language, restricting its broader applicability. Further training on diverse datasets would be essential for wider adoption. Another critical consideration for TSR systems is the privacy of the image data used for classification. TSR systems use image data to classify signs, raising potential privacy concerns related to camera data storage and usage. Protecting this data is critical to ensure ethical use. TSR systems would impact jobs significantly. While they provide reliable assistance, humans remain crucial as traffic systems evolve. TSR should augment, not replace, human roles. However, integration into autonomous driving may cause job displacement, highlighting the need for workforce transition strategies. Jobs like taxi drivers or traffic marshals may become obsolete, while demand for developers and software engineers could rise.

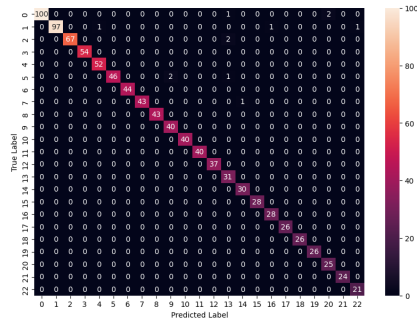
References

Khan MA, Park H, Chae J (2023) A lightweight convolutional neural network (CNN) architecture for Traffic sign recognition in urban road networks. Electronics 12:1802. doi: 10.3390/electronics12081802

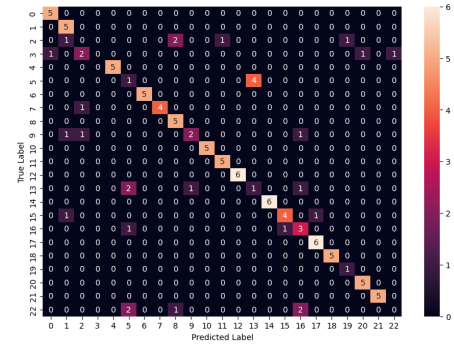
Fleyeh M (2008) Traffic and Road Sign Recognition.



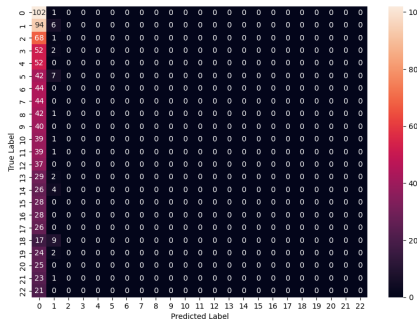
## Appendix



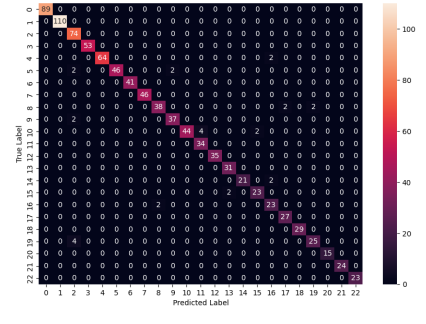
Confusion matrix of fine-tuned CNN against internal dataset



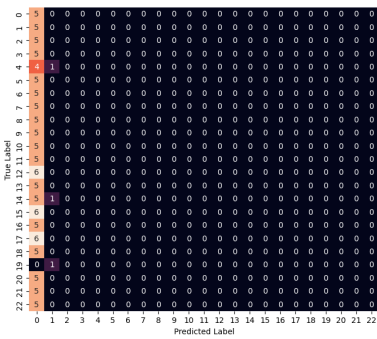
Confusion matrix of RF against external dataset



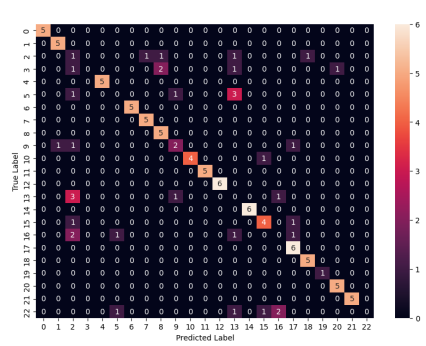
Confusion matrix of pre-trained model against internal dataset



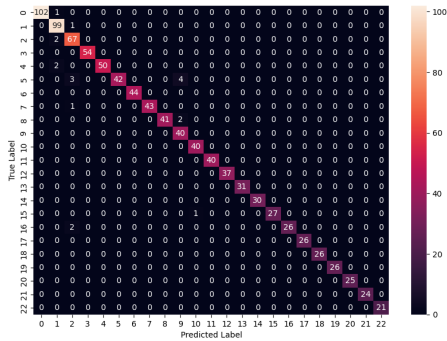
Confusion matrix of RF against internal dataset after pooling



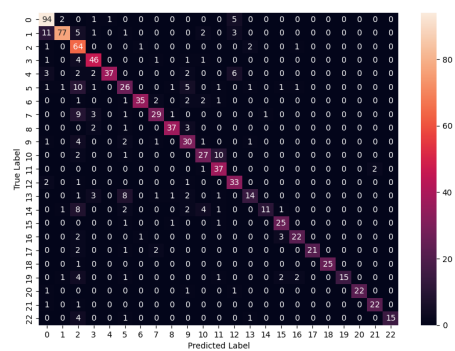
Confusion matrix of pre-trained model against external dataset



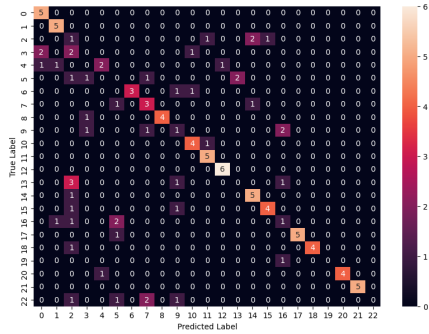
Confusion matrix of RF against external dataset after pooling



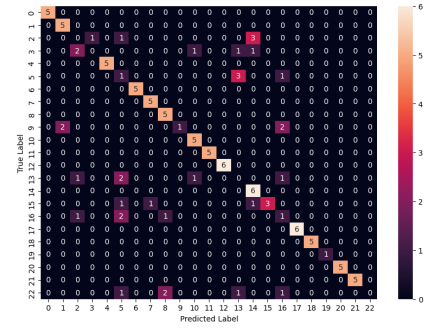
Confusion matrix of RF against internal dataset



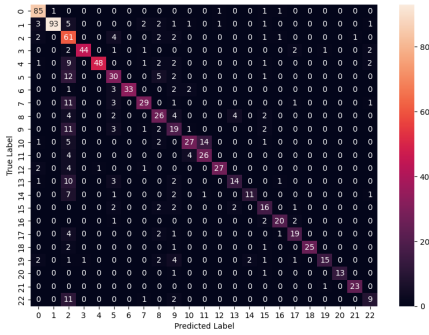
Confusion matrix of KNN model against internal dataset



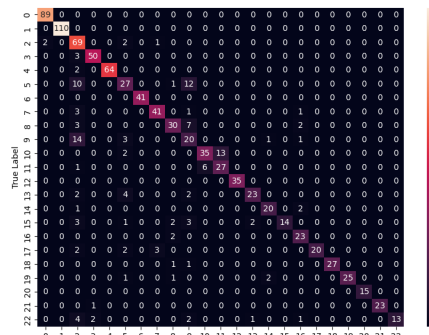
Confusion matrix of KNN model against external dataset



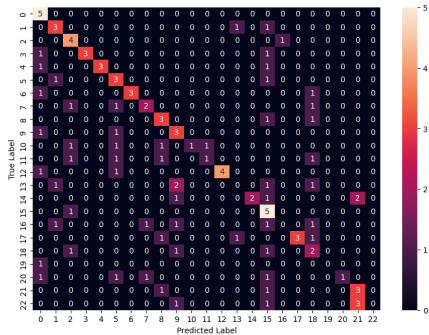
Confusion matrix of SVM model against external dataset



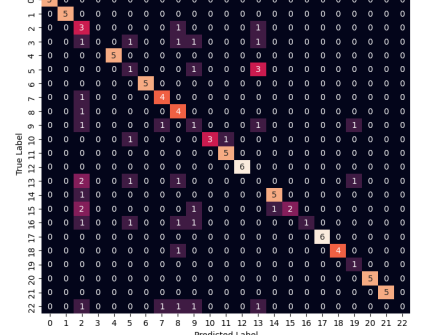
Confusion matrix of KNN against internal dataset after pooling



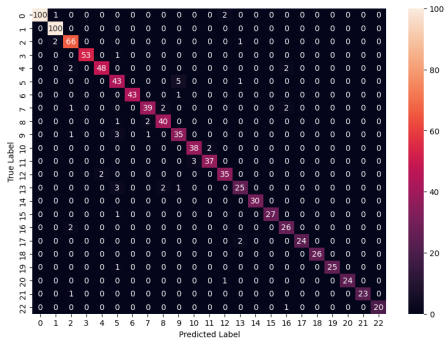
Confusion matrix of SVM model against internal dataset after pooling



Confusion matrix of KNN against external dataset after pooling



Confusion matrix of SVM model against external dataset after pooling



Confusion matrix of SVM against internal dataset